Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

CS5741
Concurrency & Parallelism in Software Development

Lecture 5 – Scheduling Techniques

Dr Faeq Alrimawi

HOST INSTITUTION
UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

PARTNER INSTITUTIONS
NUI Galway OÉ Gaillimh
Maynooth University
Trinity College Dublin
University College Dublin
UCC
MTU
DCU
Waterford Institute of Technology
Dundalk Institute of Technology
LIT
GMIT

Today, We Are Covering…

- Overview
- Static Scheduling
- Dynamic Scheduling
- Real-Time Scheduling

Overview

- **Scheduling** refers to **how tasks** are **assigned** to resources (e.g., processors) over time.
- Something called a **scheduler** takes care of this.
- Affects efficiency and performance.
- Types
  - Static
  - Dynamic
  - Real-Time

Scheduler Goals

- Maximise throughput
  - Maximise the total amount of work completed within a time period.
- Minimise wait time
  - Minimise time from when tasks are ready to execute to the first point it begins execution.
- Minimise latency or response time
  - Minimise time from when tasks are ready to execute to when they are finished or handed their first output to the user.
- Maximise fairness
  - Equal resource time to each task.

Today, We Are Covering…

- Overview
- Static Scheduling
- Dynamic Scheduling
- Real-Time Scheduling

Static Scheduling

- Tasks are **assigned** to processors **in advance.**
  - Before the program starts running.
  - Remains fixed throughout the execution.
- Characteristics
  - **Predefined Task Assignment**: tasks are mapped to processors before execution begins.
  - **Low/No Overhead**: no scheduling overhead during execution.
  - **Simple Task-to-Processor Mapping**: the scheduler knows the task set and system resources, so it can create an efficient mapping.

When to Use

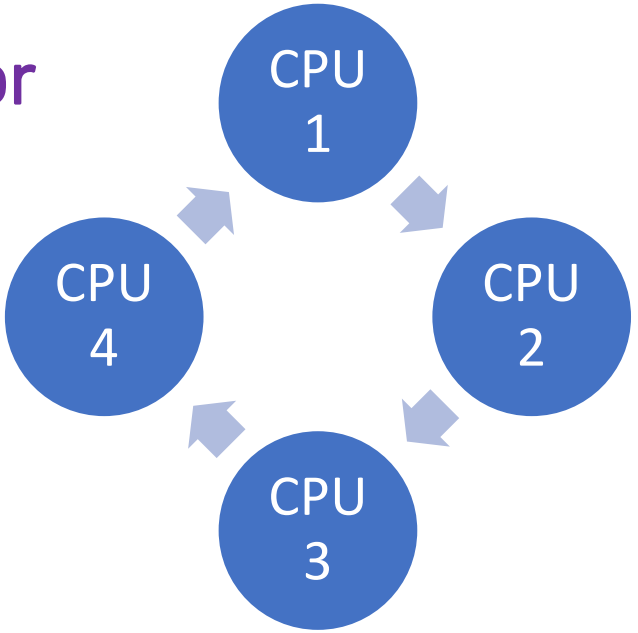
- **Regular workloads**: static scheduling works well for problems where the **workload** is **predictable** and can be evenly distributed across processors.
  - E.g., matrix multiplication in science, if there are 100 rows in the matrix and 4 processors, each processor will be assigned 25 rows in advance, ensuring a balanced load.

- **Low overhead required**: it's useful when **minimising runtime overhead** is crucial because tasks are assigned once, and the scheduler is not involved at runtime.
  - E.g., graphics rendering, if an image is 800x800 pixels, and there are 4 processors, the image can be divided into four 400x400 blocks, each assigned to a different processor.

Some Static Algorithms (1-3)

Round-Robin Scheduling

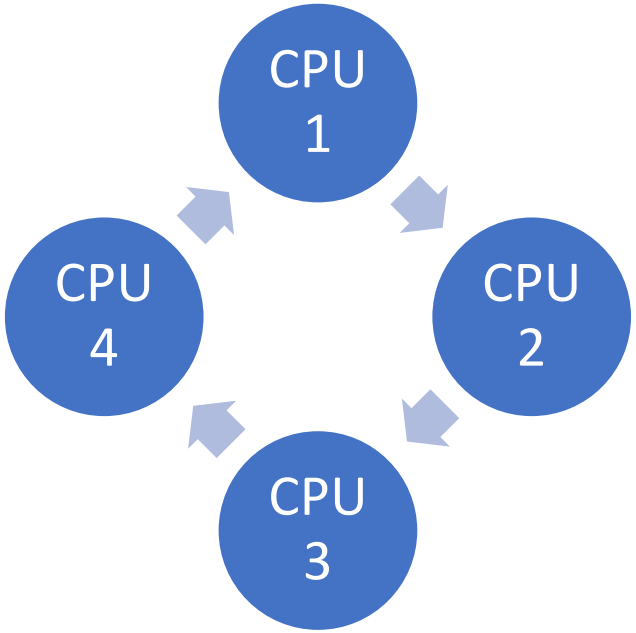
- Assigns tasks to processors in a **cyclic** manner.
- Each task is given to the **next available processor** in sequence until all tasks are distributed.
- Works for **tasks** of roughly **equal size**.
- Easy to implement.
- **However**, does not account for **varying task complexities** or processor speeds.



Some Static Algorithms (1-3)

Round-Robin Scheduling

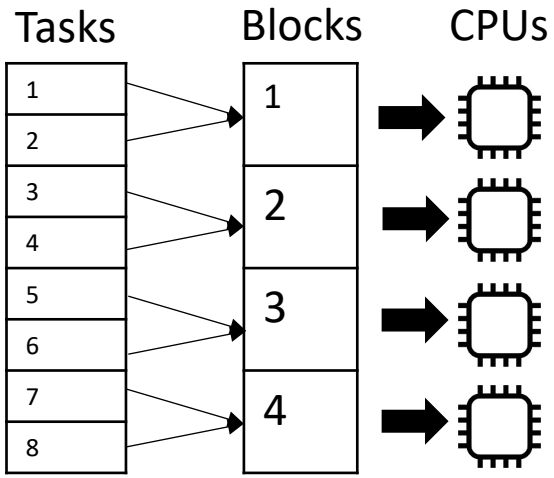
- For example, assigning multiple web server requests to different threads.
  - Processors: P1, P2, P3
  - Tasks: T1, T2, T3, T4, T5, T6
  - Assignments:
    P1 -> T1, T4
    P2 -> T2, T5
    P3 -> T3, T6



Some Static Algorithms (2-3)

Block Scheduling

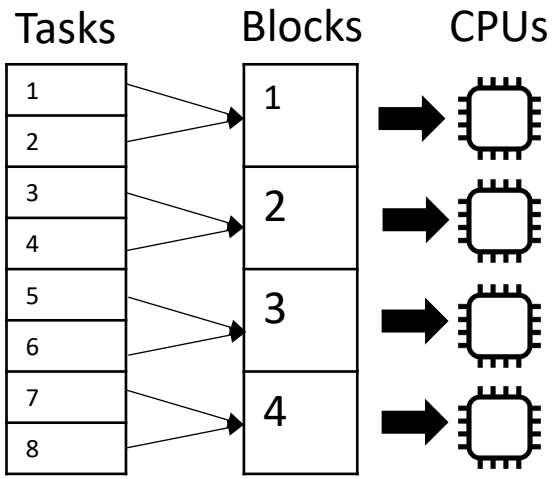
- Divide tasks into blocks normally equal to the number of processors
- Assign each block to a single processor.
- Works well when the number of tasks is a multiple of the number of processors.
- Simple to implement with minimal runtime overhead.
- However, can lead to idle processors if tasks are not evenly divisible by the number of processors



Some Static Algorithms (2-3)

Block Scheduling

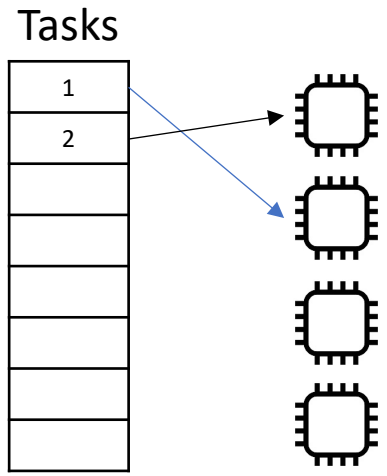
- For example, summing elements of a large array split into equal blocks.
  - Processors: P1, P2, P3
  - Array elements: [1, 2, 3, 4, 5, 6]
  - Assignment:
    P1 -> [1, 2]
    P2 -> [3, 4]
    P3 -> [5, 6]



Some Static Algorithms (3-3)

Affinity Scheduling

- Assigns tasks based on the principle of **processor/cache affinity.**
  - i.e. a task is always executed on the same processor if possible.
- Helps optimise **cache performance.**
- Works for problems where **cache performance is critical**
  - Such as loop nests in numerical simulations or scientific computations.



Static Scheduling

Advantages

- **No runtime overhead**: no scheduling decisions are made at runtime, which reduces the overhead.
- **Predictable performance**: since tasks are mapped beforehand, execution times can be predicted and optimised.

Disadvantages

- **Inflexibility**: can be inefficient if the actual runtime workload is different from what was predicted.
- **Load imbalance**: If the task sizes are uneven or if some tasks take longer to complete, some processors might finish early, leading to idle time.

Today, We Are Covering…

- Overview
- Static Scheduling
- Dynamic Scheduling
- Real-Time Scheduling

Dynamic Scheduling

- Tasks are **assigned** to processors at **runtime**.
- The **scheduler dynamically decides** which task runs on which processor.
- Characteristics
  - **Runtime Task Assignment**: the scheduling decisions are made **during program execution**.
  - **Load Balancing**: The scheduler can **balance** the **workload dynamically** by distributing tasks to processors based on their availability and current load.
  - **Adaptability**: Dynamic scheduling is **adaptive** to **varying workloads** and system conditions (e.g., heterogeneous processors, network delays).

When to Use

- **Irregular or unpredictable workloads**
  - It works well for problems where task sizes or execution times are not known in advance.
  - For example, in a parallel search algorithm (e.g., for solving a maze), the time required to search different regions might vary.

- **Systems with varying resources**
  - It’s useful when the available processors and resources can change during execution.
  - For example, cloud computing

Some Dynamic Algorithms (1-2)

- Work-Stealing Algorithm
  - A worker "steals" tasks from other busy workers' queues.
  - Used in a parallel computation where some tasks take longer to finish (e.g., different branches of a recursive computation like merge sort)
- Least Laxity First (LLF)
  - The **priority varies** according to the **difference** between the remaining **execution time** of the task instance and its relative **deadline**.
  - Used in systems where tasks must meet **strict deadlines**, such as in industrial control systems, telecommunications, and autonomous systems.

Some Dynamic Algorithms (2-2)

- Earliest Deadline First (EDF)
  - Tasks are assigned deadlines, and the scheduler dynamically **selects tasks** based on which one has the **earliest deadline**.
  - As tasks are completed or new tasks arrive, the system **continually re-evaluates the task priorities**.
  - E.g., in an embedded system controlling a **robotic arm**, different **tasks** (e.g., **motor control**, sensor readings) have **deadlines** based on when they must be executed.

Dynamic Scheduling

Advantages

- **Flexibility**: dynamic scheduling can adjust to uneven workloads and varying execution times.
- **Better load balancing**: the runtime scheduler can distribute tasks based on processor availability, ensuring that no processor remains idle.

Disadvantages

- **Runtime overhead**: since scheduling decisions are made during execution, there is additional overhead for the scheduler to continuously monitor and assign tasks.
- **Unpredictable performance**: due to the dynamic nature of scheduling, it can be harder to predict execution times and optimise performance beforehand.

Today, We Are Covering…

- Overview
- Static Scheduling
- Dynamic Scheduling
- Real-Time Scheduling

Real-Time Scheduling

- Designed for systems where tasks must be completed within **specific timing constraints**.
- Systems often **interact with the physical world** (e.g., robotics, automotive systems) and have strict deadlines for task completion.
- Characteristics
  - **Hard Real-Time**: tasks must meet **strict deadlines**. Failure to meet these deadlines could lead to system failure (e.g., a braking system in a car must engage within a specific time window).
  - **Soft Real-Time**: Tasks should **meet** their **deadlines as often as possible**, but occasional delays are tolerable (e.g., video streaming, where a few missed frames are acceptable).

When to Use

Time-sensitive applications/systems

Real-time scheduling is used in applications like embedded systems, industrial control systems, avionics, and financial trading, where failing to meet deadlines can have serious consequences.

Some Real-Time Algorithms (1-2)

- Earliest Deadline First (EDF)
  - Saw this earlier as a dynamic algorithm
- Rate-Monotonic Scheduling (RMS)
  - A priority-based real-time scheduling algorithm where **tasks with shorter periods** (higher frequency) are **given higher priority**.

Real-Time Scheduling

Advantages

- **Guaranteed Timing**: real-time scheduling ensures that tasks meet their deadlines, making it suitable for critical systems where timing is essential.
- **Predictability**: real-time systems are highly predictable and reliable in terms of task completion times.

Disadvantages

- **Complexity**: implementing real-time scheduling requires careful analysis of task timing and deadlines, making it more complex than other scheduling techniques.
- **Limited flexibility**: real-time systems often have less flexibility in handling unexpected tasks or changes in workload, as the scheduler is tightly constrained by deadlines.

Summary

Characteristic	Static Scheduling	Dynamic Scheduling	Real-Time Scheduling
Task Assignment	Predefined before runtime	Assigned at runtime	Based on deadlines and priority
Overhead	Low (no runtime scheduling decisions)	Higher (runtime decisions)	Moderate to high (must consider deadlines)
Flexibility	Inflexible (works for regular workloads)	Flexible (adaptable to irregular tasks)	Strict (must meet timing constraints)
Load Balancing	Fixed load distribution	Dynamic load balancing	Load balancing depends on task priority
Use Cases	Predictable workloads (e.g., scientific)	Unpredictable workloads (e.g., web servers)	Time-sensitive systems (e.g., robotics, automotive)

Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

What Are Your Questions?

HOST INSTITUTION

UNIVERSITY OF LIMERICK OLLSCOIL LUIMNIGH

PARTNER INSTITUTIONS

NUI Galway OÉ Gaillimh · Maynooth University · Trinity College Dublin, Coláiste na Tríonóide, Baile Átha Cliath, The University of Dublin · University College Dublin, An Coláiste Ollscoile, Baile Átha Cliath · UCC · MTU · DCU · Waterford Institute of Technology · Dundalk Institute of Technology, Dhún Dealgan · LIT · GMIT